

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Madhavan S	Reg. No.:	192425113
Section / Batch:	2024 CSE AI	Date:	18/08/2026

Code of Conduct

I, Madhavan S/192425113 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Madhavan S

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Q1. Banking Chatbot – CFG-Based Query Parsing

Scenario: A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries but faces issues such as incorrect interpretation of ambiguous sentences, failure to handle subject–verb agreement, and inefficient parsing for long queries.

Example user query: “Show me the transactions with the card from last month”

a) Ambiguity Analysis using CFG

A basic CFG for this query can be written as:

```
S → VP
VP → V NP PP PP | V NP PP
NP → Pro NP1 | Det N | Det N PP
PP → P NP
V → Show
Pro → me
Det → the
N → transactions | card | month
P → with | from
```

This grammar licenses at least two distinct parse trees for the same sentence, which is the classic PP-attachment ambiguity problem:

- Parse 1 (low attachment): “with the card” and “from last month” both modify “transactions” → [transactions [with the card] [from last month]]. Meaning: retrieve transactions that were made using the card, during last month.
- Parse 2 (nested attachment): “from last month” modifies “the card” instead of “transactions” → [transactions [with [the card [from last month]]]]. Meaning: retrieve transactions associated with a card that was issued last month.
- Parse 3 (verb attachment): “with the card” attaches to the verb “Show”, i.e. “show (using the card as a filter object)” rather than modifying “transactions” at all.

A plain CFG cannot rank these structures — all three are equally valid derivations, so the parser has no principled way to pick the intended reading. There is also lexical ambiguity in “card” (debit vs. credit vs. SIM/ID card) which CFG rules alone cannot resolve since CFG only encodes syntactic category, not word sense.

b) Limitations of CFG for Real-World Conversational Queries

- No disambiguation mechanism: CFG treats every valid derivation as equally acceptable, so it cannot select the most plausible parse among competing PP-attachment or scope readings.
- No agreement checking: plain CFG rules such as NP → Det N do not encode number/person, so “the transaction show” or “this transactions is” can be accepted as grammatical unless the grammar is manually duplicated for every agreement combination, which explodes rule count.
- Combinatorial / exponential blow-up: for long, conversational queries with multiple PPs and conjunctions, the number of derivable parse trees grows exponentially, making naive CFG parsing computationally expensive and slow for a real-time chatbot.
- No robustness to disfluency or ill-formed input: users type incomplete or ungrammatical queries (“txns card last month?”); CFG parsing fails outright instead of degrading gracefully.
- No semantic grounding: CFG only produces a syntax tree, not an interpretable intent/slot structure (e.g., intent = get_transactions, filter = card, time = last_month) that the banking backend can act on.

c) Improved Solution: PCFG + Feature Structures + Earley Parsing

A three-layer upgrade addresses each limitation:

1. Probabilistic CFG (PCFG):

Attach probabilities to each rule, learned from a corpus of annotated banking queries (e.g., logs where the intended attachment is known). For the ambiguous PP-attachment above, if “from [time-period]” attaches to “transactions” in 90%

of training examples versus attaching to “card” in 10%, the PCFG assigns a higher probability to Parse 1, allowing the parser to automatically select it via the Viterbi/CKY-style max-probability parse instead of returning an arbitrary tree.

2. Feature Structures:

Augment categories with attribute–value pairs, e.g. NP[NUM:sg, PERS:3] and V[AGR:NUM=sg]. Unification during parsing rejects derivations where subject and verb features disagree, directly solving the subject–verb agreement failures. Feature structures can also carry a SUBCAT (sub-categorization) feature on the verb, e.g. Show[SUBCAT: NP, PP*], which constrains which PP attachments are syntactically licensed for “show” versus being forced to attach to the nearest NP.

3. Earley Parsing:

Replace naive top-down/backtracking CFG parsing with the Earley chart-parsing algorithm. Earley builds a chart of partial parses (states) and reuses previously computed sub-derivations instead of recomputing them, giving worst-case $O(n^3)$ time and near-linear time for well-behaved grammars — far better than the exponential blow-up of naive backtracking on long queries. Earley also naturally produces a shared packed parse forest, which is the structure the PCFG probabilities are applied over to pick the single best parse.

Combined pipeline: tokenize → Earley parser builds a parse forest over the CFG/feature-augmented grammar → feature unification prunes agreement-violating trees → PCFG probabilities rank the remaining trees → highest-probability, feature-consistent parse is converted to an intent/slot semantic frame for the banking backend.

d) Justification of Accuracy and Efficiency Gains

- Accuracy: PCFG probabilities, trained on real banking-query data, resolve attachment and lexical ambiguity statistically rather than arbitrarily, so the chatbot consistently picks the reading a user actually intended (e.g., transactions filtered by card and month, not a card issued last month).
- Correctness: feature-structure unification eliminates ungrammatical parses caused by agreement mismatches, reducing downstream misinterpretation of user intent.
- Efficiency: Earley’s dynamic-programming chart avoids re-deriving identical sub-trees, so response latency stays low and roughly polynomial even as queries get longer — critical for a real-time customer-facing chatbot.
- Scalability & maintainability: probabilities can be re-estimated as more query logs are collected, so the system improves over time without hand-writing new CFG rules for every new sentence pattern, and sub-categorization frames make it easy to add new banking intents (loans, cards, FDs) in a structured way.
- Business impact: fewer misrouted queries reduce escalation to human agents, improving customer satisfaction and lowering support cost — a directly measurable industry benefit.

Q2. Voice Assistant – Top-Down vs. Earley Parsing

Scenario: A voice assistant (Alexa/Siri-type) processes spoken commands using top-down parsing but faces backtracking issues, poor handling of incomplete/ambiguous input, and difficulty with real-time response.

Example command: “Book a flight to Delhi with a window seat”

a) Multiple Parse Structures

The prepositional phrase “with a window seat” is attachment-ambiguous, producing at least two syntactically valid readings:

- Reading 1 (NP attachment – intended meaning): [Book [a flight to Delhi] [with a window seat]] — “with a window seat” modifies “a flight to Delhi”, i.e., book a flight that has a window seat.
- Reading 2 (VP attachment – instrumental misreading): [Book [a flight to Delhi] [with a window seat]] attaches to the verb “Book”, i.e., “use a window seat as the means of booking” — nonsensical but structurally permitted by a plain grammar.

- A further ambiguity: “to Delhi” could in principle attach to “flight” (destination of the flight) or, in a longer variant, to “Book” (destination of the booking action) — the grammar alone doesn’t distinguish these without semantic/selectional restrictions.

Because the grammar admits both bracketings, a naive parser returns multiple parse trees for one utterance, and without ranking, the assistant cannot reliably know the user wants a window-seat flight rather than some other reading.

b) Limitations of Top-Down Parsing in Real-Time Voice Applications

- Left-recursion failure: top-down parsers loop infinitely on left-recursive rules (e.g., $NP \rightarrow NP PP$), which are common and natural for describing nested modifiers in spoken commands.
- Exponential backtracking: when a chosen expansion fails later in the sentence, the parser must backtrack and try alternative rules, and for ambiguous/long commands this branching can grow exponentially, causing unacceptable latency.
- No reuse of partial work: each backtrack re-parses sub-phrases from scratch instead of reusing previously built structure, wasting computation — costly for a device with limited on-board compute (smart speakers, phones).
- Poor handling of incomplete input: streaming speech recognition delivers words incrementally; top-down parsing generally needs to commit to a full top-level rule expansion before it can confirm a partial structure, so it handles partial/incomplete utterances (pauses, self-corrections, “uh, to Delhi— I mean Mumbai”) poorly.
- Sensitive to noise: any ASR (speech-to-text) misrecognition on a single word can cause the entire top-down derivation to fail with no graceful fallback, which is a real risk for accented or noisy speech input.

c) How Earley Parsing Improves Handling of Ambiguity and Partial Input

- Chart-based dynamic programming: Earley maintains a chart of states per input position and reuses completed sub-derivations, eliminating the redundant recomputation that plagues backtracking.
- Handles left recursion natively: because it works left-to-right building states rather than expanding rules depth-first, Earley parses left-recursive grammars correctly without looping.
- Ambiguity as a packed forest, not branching search: instead of exploring each ambiguous reading as a separate backtracked path, Earley represents all valid parses compactly in one chart, and a PCFG layer on top selects the most probable one — avoiding the combinatorial explosion.
- Genuine incremental processing: the Earley chart at input position i is a valid, well-defined partial state; this lets the assistant start constraining/predicting structure as soon as each word arrives from the streaming ASR, rather than waiting for the full utterance and risking a full re-parse from scratch, which better matches how voice input streams in.
- Graceful degradation: if the utterance is incomplete or the user pauses, the chart still holds all valid partial analyses so far, letting the system respond faster or ask a targeted clarifying question about the specific ambiguous slot (e.g., “did you mean the flight has a window seat?”).

d) Top-Down vs. Earley Parsing – Performance Comparison

Criterion	Top-Down Parsing	Earley Parsing
Worst-case time complexity	Exponential ($O(2^n)$) with backtracking on ambiguous grammars	Polynomial, $O(n^3)$ worst case; near $O(n)$ for many practical grammars
Left recursion	Fails / infinite loop	Handled natively
Reuse of sub-parses	None – recomputes on backtrack	Yes – chart reuses completed states
Ambiguity handling	Separate backtracked branches per reading	Single shared packed parse forest
Incremental / streaming input	Poor – needs full rule commitment	Well suited – valid partial chart at each step

Robustness to noisy ASR	Whole derivation fails on one bad token	Localized failure; other chart states remain usable
Suitability for real-time voice assistants	Weak – unpredictable latency spikes	Strong – predictable, scalable latency

Overall, Earley parsing is the more industry-appropriate choice for a real-time voice assistant: it converts the unpredictable, exponential worst case of top-down backtracking into a predictable polynomial-time chart computation, natively supports the incremental nature of streaming speech, and — when paired with PCFG probabilities — resolves attachment ambiguities like “with a window seat” in favor of the statistically likely, user-intended reading.

Q3. Healthcare Report Analysis – Integrated Semantic Architecture

Scenario: A healthcare company needs an AI system that extracts diagnosis, prescribed treatments, and follow-up actions from patient reports, handling complex/lengthy sentences, structural ambiguity, subject–verb agreement, and relationships between medical actions and entities (sub-categorization), in real time.

Example input: “The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

a) Integrated NLP Architecture Design

The architecture is organized as a layered pipeline so each technique addresses a specific weakness of using CFG alone:

Layer	Component	Role
1	Preprocessing	Tokenization, sentence splitting, POS tagging, named-entity recognition (doctor, patient, medication, Chennai)
2	CFG Layer	Builds the base syntactic skeleton – NP/VP/PP structure, relative clauses, coordination
3	PCFG Layer	Assigns learned probabilities over CFG rule expansions to rank competing parses (e.g., resolve attachment of “last week” and “in Chennai”)
4	Feature-Structure Layer	Unification for agreement (doctor ... recommends = 3rd-person singular) and for sub-categorization (verb-specific complement types)
5	Efficient Parser (Earley/Chart)	Polynomial-time parsing of long, nested sentences with relative clauses and coordination; produces a packed parse forest
6	Semantic Interpretation	Maps the selected parse to predicate-calculus / semantic-role representation (agent, action, theme, location)
7	Structured Output	Emits diagnosis / action / location fields (e.g., JSON) for the hospital’s EHR system

Data flows top to bottom; each layer narrows the space of candidate structures handed to the next, so the architecture is both accurate (probabilistic + feature-based disambiguation) and tractable (efficient parsing) on long clinical sentences.

b) Step-by-Step Workflow for the Given Sentence

- Step 1 – Preprocessing: tokenize and POS-tag the sentence; NER tags “doctor” and “patient” as PERSON/ROLE entities, “medication” as TREATMENT, “Chennai” as LOCATION.

- Step 2 – CFG parse of clause structure: identifies the main clause “The doctor ... recommends X” and the embedded relative clause “who reviewed the patient last week” modifying “the doctor”, using a rule such as NP → Det N RelClause and RelClause → who VP.
- Step 3 – PCFG disambiguation: resolves the temporal-ad adjunct attachment of “last week” — the learned probabilities favor attaching it to “reviewed” (inside the relative clause) rather than to “recommends”, matching the intended reading that the review happened last week, not the recommendation.
- Step 4 – Feature unification for agreement: the head noun “doctor” carries [NUM:sg, PERS:3]; unification confirms the main verb “recommends” correctly agrees ([AGR:3sg]), rejecting any competing parse that would mismatch agreement (e.g., misreading “patient” as the subject).
- Step 5 – Sub-categorization / coordination handling: the verb “recommends” is looked up with SUBCAT: VP-gerund-coordination, licensing a coordinated gerund complement — “starting medication” and “scheduling a follow-up visit” — as two coordinated actions the doctor recommends, rather than two independent top-level clauses.
- Step 6 – PP attachment: “in Chennai” is attached to “a follow-up visit” (not to “scheduling” in general or to the whole sentence) using the sub-categorization frame of “visit”, which expects an optional location modifier.
- Step 7 – Efficient (Earley) parsing ties Steps 2–6 together: the chart parser builds all these nested structures — relative clause, coordination, PP attachment — in polynomial time even though the sentence is long, avoiding the exponential cost a naive backtracking parser would incur.
- Step 8 – Semantic interpretation: the selected parse is converted to a predicate-calculus-style representation, e.g.:

```
Doctor(d1) ∧ Patient(p1) ∧ Reviewed(d1, p1, time: last_week)
∧ Recommends(d1,
    Start(medication),
    Schedule(follow_up_visit, location: Chennai))
```
- Step 9 – Structured output: the representation is flattened into hospital-system fields, e.g. { "diagnosis_context": "reviewed_by_doctor", "actions": ["start_medication", "schedule_follow_up"], "location": "Chennai", "timeframe": "last_week" } for storage in the patient’s EHR.

c) Method for Sub-Categorization, Real-Time Processing, and Scalability

Sub-categorization frames for medical verbs:

Build a domain-specific sub-categorization lexicon that records the allowed complement patterns for common clinical verbs, so the parser/feature-unifier can accept correct structures and reject implausible ones automatically:

Verb	Sub-categorization frame	Example
recommend	NP-Subj, VP-gerund (optionally coordinated)	recommends starting medication and scheduling a visit
prescribe	NP-Subj, NP-Obj (medication), [PP-dosage]	prescribed 500mg amoxicillin
schedule	NP-Subj, NP-Obj (event), [PP-loc], [PP-time]	scheduling a follow-up visit in Chennai
review	NP-Subj, NP-Obj (patient/report), [PP-time]	reviewed the patient last week

Each frame is attached as a feature (SUBCAT) on the verb’s lexical entry; during parsing, feature unification only admits attachments/coordinations consistent with the verb’s frame, which directly resolves relationship extraction between medical actions and entities and prevents the mis-attachment errors a bare CFG would allow.

Real-time processing:

- Use Earley/chart parsing (polynomial time) instead of naive backtracking CFG parsing so long clinical sentences do not cause latency spikes.
- Process incrementally per sentence/clause so the system can start extracting entities while the rest of a long report is still being parsed.

- Cache and reuse PCFG parse forests for recurring sentence templates common in clinical notes (e.g., standard discharge-summary phrasing).
- Batch probability scoring on GPU for high-throughput hospital workloads (many reports per minute).

Improving accuracy and scalability:

- Retrain PCFG probabilities periodically on newly annotated hospital reports so rule/attachment preferences stay current with real clinical language.
- Keep the architecture modular (separate CFG/PCFG grammar, feature/sub-categorization lexicon, and semantic-mapping layer) so new medical specialties or verb frames can be added without redesigning the whole pipeline.
- Route low-confidence parses (below a PCFG probability threshold) to a human-in-the-loop review queue rather than forcing an automatic decision, which protects accuracy on unusual or highly complex sentences.
- Deploy the pipeline as independently scalable microservices (parsing, feature unification, semantic mapping) so hospital-wide load can be handled by scaling only the bottleneck stage.